Truth — Project Overview

1. Application Overview

Truth is a modular, local-first personal intelligence and software platform designed to help users understand their circumstances, organise their goals, reason through decisions, model connected systems, and coordinate actions through controlled software interfaces. Its central intelligence layer, the Truth Oracle, interprets requests, retrieves relevant information, distinguishes evidence from assumptions, evaluates options, and produces recommendations. The Personal Interface (PI) coordinates user goals, tasks, preferences, approvals, and progress. Beneath these experiences, the Truth Kernel governs state changes, the Truth Runtime coordinates components and task execution, the Truth Intermediate Representation (TIR) provides shared data contracts between modules, the Compute Engine performs deterministic calculations, and the persistence and network layers maintain records, relationships, dependencies, and provenance. Truth should initially operate as an application on existing operating systems, with a long-term goal of local-first autonomy and progressively greater self-contained operation where device capabilities permit. It must remain modular, auditable, permission-controlled, recoverable, and extensible without requiring every planned capability to be built in the first release.

2. Project Goals

- Establish a reliable software foundation. Define and implement clear contracts, module boundaries, lifecycle rules, error handling, and tests before adding complex product behaviour.
- Build the Truth Runtime. Initialise registered components, resolve dependencies, dispatch tasks and events, manage execution limits, track outcomes, and shut down safely.
- Build the Truth Kernel. Process validated observations, coordinate state transitions, retrieve relevant memory, invoke reasoning, and commit accepted outcomes through controlled interfaces.
- Establish TIR as the shared internal representation. Give system modules a consistent, versioned way to exchange entities, relationships, observations, events, states, commands, tasks, capabilities, permissions, results, evidence references, and transformation records.
- Separate deterministic computation from AI reasoning. Use explicit calculation procedures and versioned rules for numerical operations and dependency resolution. Use AI models for interpretation, planning, and other appropriate reasoning tasks without treating generated text as verified computation.

- Implement reliable local persistence. Store events, current state, and memory records with timestamps, stable identifiers, provenance, validation, and recovery support.
- Build a Network Registry and Dependency Matrix. Represent entities and their typed relationships separately from operational dependencies, allowing the system to trace connections, resolve prerequisites, and analyse the impact of changes.
- Build the Truth Oracle and PI. Help users interpret information, explore options, establish goals, plan tasks, request approvals, and track progress against measurable outcomes.
- Enforce security and governed execution. Require appropriate permissions for operations, protect credentials and private data, validate untrusted inputs, record important actions, and provide containment and recovery procedures.
- Deliver a usable frontend and administrative interface. Provide a user-facing application for interacting with Truth and a separately governed admin console for system operations, monitoring, access control, and auditing.
- Support progressive local autonomy. Design interfaces and boundaries that can support offline and local-first operation, followed by carefully governed integrations and additional autonomy as the system matures.
- Make every development phase verifiable. Require working code, automated tests, documented contracts, and observable acceptance criteria for each implemented capability.

3. Core User Flow

The following flow describes the intended end-to-end behaviour. Some steps will be introduced incrementally and are not assumed to exist in the initial prototype.

- **User opens Truth.** The application loads its interface, checks the availability of required services, and retrieves the permitted local session, preferences, current state, and outstanding tasks.
- **Truth establishes the operating context.** The system determines which components and integrations are available, whether it is operating offline or connected, and which permissions apply to the current request.
- **User submits a request or goal.** The request may be a question, an instruction, a reflection, a calculation, a planning request, or a request to coordinate a supported operation.
- **Kernel validates the input.** The system validates the request structure, assigns identifiers for tracking, applies input limits, and rejects malformed or unsupported requests with a clear error.
- **Truth creates an observation.** The raw input is preserved as an observation with its source and relevant metadata. The system must not automatically treat every statement as a verified fact.
- **Truth retrieves relevant context.** The retrieval layer searches permitted memory records, current state, historical events, network relationships, and available evidence relevant to the request.

- **Truth constructs the reasoning context.** The Oracle receives the validated request, relevant evidence, known constraints, current state references, and applicable permissions. Missing information and conflicting evidence are identified where possible.
- **Truth selects an appropriate reasoning method.** The request is routed to retrieval, interpretation, deterministic calculation, scenario analysis, planning, or another registered capability supported by the system.
- **Truth produces a structured result.** The result contains the answer or proposed plan, relevant evidence, assumptions, uncertainties, limitations, and recommended next steps where applicable.
- **Truth evaluates proposed actions.** If the request requires an operation, the system checks the registered capability, required permissions, policy conditions, resource limits, and approval requirements.
- **Truth requests approval when necessary.** Actions that require user consent or elevated authority must wait for approval. A recommendation from the Oracle does not itself authorise execution.
- **Runtime dispatches authorised work.** The Runtime invokes only registered handlers through validated interfaces, tracks execution status, enforces applicable limits, and records failures or timeouts.
- **Truth verifies the outcome.** The system checks the returned result against the expected outcome. It distinguishes a confirmed result from an attempted action or an uncertain external response.
- **Truth records accepted changes.** Validated events, calculation results, task outcomes, and approved state changes are persisted with relevant timestamps, identifiers, and provenance. Failed or rejected operations must not be recorded as successful.
- **Truth responds to the user.** The interface presents the result, current status, relevant explanation, and any unresolved issue or next action.
- **Truth continues from the updated state.** The user can correct information, provide feedback, revise goals, continue a task, or request another operation. Future personalisation must follow the applicable privacy and permission rules.

4. Features

4.1 User Interface

- Responsive user-facing application for supported desktop and mobile screen sizes.
- Home dashboard showing relevant goals, tasks, recent activity, and system status.
- Oracle interface for questions, interpretations, planning, and supported calculations.
- Goal management with measurable success criteria, priorities, target dates, and progress evidence.
- Activity history showing task status and relevant recorded events.
- Settings for user preferences, privacy controls, connected services, and supported operating modes.

- Accessible navigation, clear loading and error states, and light and dark themes.

4.2 Truth Kernel

- Validated request processing.
- Observation creation and context retrieval.
- Explicit state-transition rules.
- Coordination of reasoning and response generation.
- Controlled persistence and commit boundaries.
- Correlation identifiers for tracing a request through the system.
- Defined behaviour for invalid input, missing context, component failure, and interrupted operations.

4.3 Truth Runtime

- Component registration and lifecycle management.
- Dependency validation and initialisation ordering.
- Task and event dispatch.
- Task status tracking, including rejected, failed, cancelled, timed-out, and approval-pending tasks.
- Execution timeouts, resource limits, bounded retries, and cancellation where supported.
- Health checks, structured logs, diagnostics, and graceful shutdown.
- Safe startup and recovery procedures.
- Explicit distinction between commands that request work and events that record something that occurred.

4.4 Truth Intermediate Representation (TIR)

- Versioned, language-neutral data contracts.
- Canonical representations for observations, entities, relationships, events, states, tasks, commands, capabilities, permissions, results, evidence references, and transformation records.
- Schema validation and compatibility checks.
- Stable identifiers and correlation identifiers.
- Provenance tracking and explicit uncertainty where applicable.
- Registered adapters for supported external systems.
- Transformation records describing input, output, version, and known information loss.
- A controlled handoff from representation and validation to execution. *(Note: TIR is an interoperability contract, not a promise to translate every application, programming language, file format, or executable automatically.)*

4.5 Compute Engine

- Versioned registry of supported calculations and rules.

- Input validation, including type, range, unit, and precision checks where relevant.
- Deterministic arithmetic and other explicitly implemented calculations.
- Dependency resolution and prerequisite checking.
- Directed dependency graphs with cycle detection where applicable.
- Scenario calculations isolated from committed production state.
- Structured calculation results, error codes, and reproducible inputs.
- Calculation provenance, including procedure version and input references.
- Verification before results are committed to state.

4.6 State, Memory, and Persistence

- Local persistence for supported operating environments.
- Separate records for observations, historical events, current state, memory, inferences, commands, and results.
- Event time and record time stored separately.
- Versioned state snapshots and controlled state updates.
- Retrieval based on relevance, source, time, confidence, sensitivity, and correction status.
- Duplicate detection and explicit correction or supersession of records.
- Database consistency checks, restart recovery, and rebuildable search indexes.
- User controls for inspecting, exporting, correcting, and deleting eligible personal records.
- Appropriate retention rules for audit and security records.

4.7 Network Registry and Dependency Matrix

- Stable entity identifiers independent of display names.
- Typed relationships between entities.
- Relationship provenance, validity periods, and uncertainty where applicable.
- Operational dependencies between tasks, services, calculations, data, resources, and policies.
- Dependency validation and cycle detection according to the dependency type.
- Prerequisite resolution and execution ordering.
- Impact analysis showing directly affected, transitively affected, uncertain, unaffected, and blocked components where determinable.
- Historical tracking of changes to registered entities and dependencies.
- Integration with TIR, persistence, the Kernel, Runtime, Compute Engine, and Oracle.

4.8 Truth Oracle

- Retrieval and interpretation of relevant context.
- Structured reasoning requests and results.
- Separation of verified information, user-reported information, external claims, model-generated interpretations, hypotheses, and unknowns.
- Explicit assumptions, evidence references, uncertainty, and limitations.
- Option generation and comparison.

- Scenario analysis using deterministic calculations when required.
- Planning based on prerequisites, constraints, and success criteria.
- Replaceable AI provider adapters behind stable internal contracts.
- User feedback and correction mechanisms.
- No unrestricted ability to modify system state or execute external actions.

4.9 Personal Interface (PI) and Goal Management

- Goal records containing desired outcomes and measurable success criteria.
- Tasks linked to goals, prerequisites, deadlines, and responsible components.
- Progress updates supported by recorded evidence.
- Task prioritisation and identification of blockers.
- Approval requests and action status.
- User preference management with appropriate privacy controls.
- Planning across short-term and longer-term milestones.
- Clear distinction between a proposed plan, an approved plan, an attempted action, and a verified outcome.

4.10 Security, Permissions, and Recovery

- Authentication integration appropriate to the deployment environment.
- Role-based access control and scoped permissions.
- Capability registration separate from permission grants.
- Permission checks at the execution boundary.
- Least-privilege access and time- or scope-limited grants where appropriate.
- Validation of external inputs and model-generated tool requests.
- Protected credential handling and redaction of secrets from logs.
- Structured audit records for important security and administrative actions.
- Detection and recording of defined suspicious events.
- Restricted operation and containment procedures for supported environments.
- Backup, restore, integrity validation, and post-recovery checks.
- Explicit user approval for actions that exceed the current authority of the system.

4.11 Administrative Console

- Separate administrative interface for authorised operators.
- System health and component status.
- Runtime task inspection and failure diagnosis.
- Approval queue for operations requiring administrative or user authorisation.
- User, role, permission, and session management according to the operator's authority.
- Component and integration inventory.
- Audit log search, filtering, and export.
- Security incident inspection and supported containment actions.
- Configuration management with validation and audit history.

- Server-enforced authorisation for every privileged operation. *(Note: The admin console must not grant authority merely because a user can access its frontend.)*

4.12 Local-First Operation and Integrations

- A host adapter that isolates platform-specific behaviour from core logic.
- Explicit interfaces for storage, model providers, and external services.
- Offline behaviour for capabilities whose required data and compute are available locally.
- Connected operation when authorised services are available.
- Graceful handling of unavailable services and interrupted connections.
- Duplicate-safe handling of retried external operations where supported.
- A design that can evolve toward greater local autonomy without requiring an immediate custom operating system. *(Note: Actual offline support depends on the specific capability, model, storage, and device resources available.)*

4.13 Future Expansion

The architecture should preserve clear extension points for future work such as:

- More advanced simulation and network analysis.
- Additional registered application integrations.
- Governed software agents and delegated tasks.
- More extensive historical, geographical, and temporal knowledge networks.
- Personalised long-term planning and reflection.
- Regionally contextualised Truth World experiences.
- Future financial, marketplace, and company-operation systems. *(Note: These are future directions, not claims that these capabilities are part of the first working release.)*

5. In Scope

5.1 Initial Engineering Scope

- Audit the actual repository and reconcile local files with the remote repository before modifying existing work.
- Establish a clear source tree, setup instructions, configuration rules, and test commands.
- Define and version shared data contracts.
- Implement a minimal Runtime and Kernel with explicit module boundaries.
- Implement request validation, observations, structured responses, and controlled state transitions.
- Introduce local transactional persistence for the required initial records.
- Implement a minimal TIR schema and validation path.
- Implement a small deterministic calculation capability with tests.
- Implement dependency resolution and cycle detection for supported dependency types.

- Implement a minimal memory retrieval mechanism.
- Introduce capability registration and a policy-checking boundary before executable operations.
- Add structured errors, logs, correlation identifiers, and provenance.
- Provide automated unit and integration tests for the core request lifecycle.
- Document component responsibilities, interfaces, setup, and known limitations.

5.2 Initial Product Scope

- A working application shell and navigation.
- An Oracle interface connected to the implemented backend contracts.
- Basic goal and task records.
- Activity and task status views.
- Basic system health information.
- A minimal administrative interface for the operational capabilities actually implemented.
- Clear loading, empty, success, error, and permission-denied states.
- Basic privacy and settings controls appropriate to the available features.

5.3 Engineering Rules

- Inspect the existing code before choosing or replacing the stack.
- Preserve useful existing work unless an explicit, reviewed change is approved.
- Do not introduce an endpoint, module, service, or feature without defining its contract and purpose.
- Do not present planned behaviour as implemented behaviour.
- Do not treat generated AI output as verified evidence or deterministic calculation.
- Do not execute unregistered handlers or bypass permission checks.
- Require tests for critical logic and security boundaries.
- Keep the first release small enough to run and verify before expanding it.
- Track decisions, assumptions, dependencies, open questions, and acceptance criteria.
- Implement the platform incrementally, with each stage leaving a testable working baseline.

6. Out of Scope

- A custom operating system or replacement for Android, Windows, Linux, or macOS.
- A universal binary translator or universal application interpreter.
- Unrestricted autonomous access to the device or internet.
- A complete self-running company.
- A fully self-contained AI system running entirely on every user's phone.
- A universal model of all human knowledge or the entire world.
- Automatic discovery of every real-world relationship or dependency.
- A custom AI model or custom neural network framework.

- Guaranteed factual correctness or perfect predictions.
- A fully distributed compute cluster.
- Universal integration with every social media platform, application, device, or API.
- An unrestricted cyber offensive or autonomous hacking system.
- Automatic financial transactions, currency issuance, or a production Truecoin economy.
- A complete Truth World virtual civilisation or real-time global simulation.
- A complete historical, geographical, religious, or civilisational knowledge graph.
- An unrestricted agent that changes its own permissions, rewrites its own trusted code, or bypasses governance.
- Unverified production deployment.

7. Success Criteria

7.1 Repository and Setup

- A clean checkout can be configured using documented instructions.
- Required dependencies and supported runtime versions are explicitly declared.
- The application starts using the documented command.
- The test suite runs using the documented command.
- The repository contains the code and configuration required for the documented baseline.
- No required file is referenced in setup instructions without being present or explicitly documented as generated.

7.2 Kernel and Runtime

- A valid request can pass through input validation, observation creation, context retrieval, reasoning, response generation, and controlled persistence.
- An invalid request is rejected with a structured error.
- Every tracked request has identifiers sufficient to correlate its relevant operations.
- Tasks expose explicit statuses and timestamps.
- Registered components initialise in a valid dependency order.
- Missing dependencies, timeouts, cancellation, and component failures have defined outcomes.
- Shutdown and restart behaviour are tested for the supported environment.

7.3 TIR and Interoperability

- Initial TIR object types have versioned schemas.
- Invalid payloads fail validation before execution.
- Supported transformations define their input and output contracts.
- Transformation provenance and correlation identifiers are preserved where applicable.
- A mock adapter demonstrates a complete request and result path.

- Tests cover valid payloads, invalid payloads, unsupported versions, and transformation failures.

7.4 Compute Engine

- At least one deterministic calculation runs from validated inputs to a structured result.
- The same supported inputs, rules, and versions produce the same result.
- Missing prerequisites and invalid inputs produce explicit errors.
- Supported dependency cycles are detected.
- Calculation results record the procedure version and relevant input provenance.
- Automated tests verify normal cases, boundary cases, invalid inputs, and failure cases.

7.5 State, Memory, and Network Registry

- Events and current state are stored as distinct record types.
- Event time and record time can be distinguished.
- State changes pass through the defined control boundary.
- Restarting the application does not silently discard committed records.
- Duplicate handling and correction behaviour are documented and tested.
- Entity identifiers and relationship types are validated.
- Dependency resolution reports missing prerequisites and supported cycle errors.
- The system can produce a basic dependency impact report for a defined test graph.

7.6 Oracle and PI

- The Oracle accepts a structured request and returns a structured result.
- The response distinguishes evidence, assumptions, inferences, uncertainty, and unknowns where relevant.
- A basic goal can be created with success criteria and linked tasks.
- Task and goal progress is reflected in the interface using persisted data.
- Recommendations are not treated as authorisation to execute actions.
- The reasoning provider can be replaced behind its defined adapter contract without redesigning the Kernel.

7.7 Security and Recovery

- Unauthorised operations are denied at the backend execution boundary.
- Capabilities and permission grants are represented separately.
- External inputs and model-generated requests are validated before use.
- Secrets are not exposed in ordinary logs.
- Important privileged actions produce structured audit records.
- The supported backup and restore procedure can be exercised and verified.
- Recovery does not silently mark failed or unverified operations as successful.
- Security-critical paths have automated tests.

7.8 Frontend and Administration

- Core screens load using the defined application contracts.
- Loading, empty, success, error, and access-denied states are implemented where relevant.
- Forms validate input and communicate actionable errors.
- Administrative actions are authorised by the server.
- Audit records and task status are displayed accurately from actual system data.
- The interface supports the defined responsive layouts and keyboard-accessible navigation.
- The main user journey can be completed in an automated end-to-end test for the implemented features.

7.9 Delivery and Verification

- Each implemented feature maps to a requirement and an acceptance test.
- Automated tests pass in the declared development environment.
- Critical architecture decisions and unresolved risks are documented.
- No feature is marked complete solely because code has been generated.
- Repository changes are reviewed and committed through an agreed workflow.
- The team can demonstrate the working application from a clean setup.
- The project status clearly separates implemented, tested, partially implemented, planned, and blocked capabilities.

8. Implementation Order

1. **Repository audit and bootstrap:** verify the real source tree, runtime, dependencies, tests, and existing work.
2. **Shared contracts and policy boundary:** define initial schemas, identifiers, errors, component contracts, capability registration, and permission checks.
3. **Runtime and Kernel vertical slice:** make one request travel through the core lifecycle with observable status and structured output.
4. **Persistence and memory:** store events, current state, and retrievable records with provenance and restart recovery.
5. **TIR and Compute Engine:** validate shared objects, execute a deterministic calculation, and resolve a small dependency graph.
6. **Network Registry:** add typed entities, relationships, dependency traversal, and impact analysis.
7. **Oracle and PI:** introduce structured reasoning, basic goals, task planning, approvals, and progress tracking.
8. **User-facing frontend:** connect the implemented services to the primary user journey.

9. **Admin console:** implement operational monitoring, task inspection, approvals, audit views, and authorised administration.
10. **Security hardening and recovery:** test permission boundaries, failure handling, audit behaviour, backup, restore, and supported containment.
11. **Incremental expansion:** add integrations and advanced capabilities after their contracts, permissions, tests, and acceptance criteria are defined.

9. Rules for the AI Coding Agent

- Read this file and the relevant detailed PRDs before implementing a task.
- Inspect the existing repository and report the actual stack and source tree before proposing destructive changes.
- Do not assume a planned component already exists.
- Do not replace the architecture, data contracts, or technology stack without explaining the reason and obtaining approval where required.
- Work on one bounded, testable task at a time.
- Before coding, identify the relevant requirements, dependencies, expected files, and acceptance tests.
- After coding, run the available relevant tests and report the actual commands and results.
- If a command cannot be run, state why. Never fabricate a passing test.
- Keep implementation, documentation, and tests consistent.
- Do not store secrets in source code or commit private user data.
- Do not bypass policy checks for convenience or use mock results as proof of production integration.
- Do not mark work complete when required acceptance criteria remain unverified.
- Report changed files, architectural decisions, known limitations, outstanding failures, and the next recommended task.
- Preserve unrelated work and request guidance before making irreversible changes.
- Prefer simple, explicit, testable implementations over premature abstraction.

10. Relationship to the Detailed PRDs

This overview consolidates the direction of the following project specifications:

- PRD-001: Host and Local Autonomy Architecture
- PRD-002: Runtime and Execution Foundation
- PRD-003: Truth Intermediate Representation and Interoperability Layer
- PRD-004: Compute Engine and Deterministic Reasoning
- PRD-005: State, Memory, and Event Persistence
- PRD-006: Network Registry and Dependency Matrix
- PRD-007: Truth Oracle and Personal Interface
- PRD-008: Security, Permissions, Threat Containment, and Recovery

- Frontend Specification: User-facing application
- Admin Panel Frontend Specification: Administrative operations and governance

11. Definition of Done

A task is done only when its agreed acceptance criteria are satisfied, its behaviour is implemented, relevant tests have been run, documentation is updated, and the results are reported honestly. A phase is done only when its required tasks pass their acceptance criteria and the resulting system remains usable. A generated document, mock interface, untested code sample, or successful code-generation response alone is not proof of a working feature.